MARYMOUNT
UNIVERSITY

# RECEIPT SCANNER

## 3/12/2026

Marymount University

Dr. Natalia Bell

IT-548-A

James Green, Chris Duckers, Numi Tesfay

This Page Intentionally Left Blank

# RECEIPT SCANNER

This document provides an overview of the Receipt Scanner application software requirements, design, and usage. This document is produced for IT-548-A, Marymount University, as a group project. Volume 3 of this document is produced and stored externally.

Planning Document Completion Date:3/14/2026

Software Requirements Completion Date:

Software Design Completion Date:

Software Implementation Completion Date:

Software Usage Guide Completion Date:

## ORGANIZATION

This document is organized into the following sections:

**Volume-1**: Receipt Scanner Planning Document

**Volume-2**: Receipt Scanner Software Requirements

**Volume-3**: Receipt Scanner Software Design Description

**Volume-4**: Receipt Scanner code

**Volume-5**: Receipt Scanner Usage Guide

# SOFTWARE PLANNING

## RECEIPT SCANNR VOLUME 1

This Page Intentionally Left Blank

# CHAPTER 1 PROJECT OVERVIEW

## 1.1 Purpose

The purpose of this project is to explore the use of modern software development tools and artificial intelligence techniques within a Python-based application to solve a practical real-world problem. Specifically, the system aims to demonstrate how Optical Character Recognition (OCR) and automated information extraction techniques can be used to digitize paper receipts and derive meaningful financial information from them.

Many individuals and small businesses rely on paper receipts to track expenses. Manual entry of receipt data into spreadsheets or financial systems is time-consuming and prone to human error. As businesses accumulate large numbers of receipts, organizing and analyzing expense information becomes inefficient.

The Receipt Scanner application will allow a user to upload images of receipts and automatically extract key fields such as transaction date, vendor name, price paid, and gallons purchased. The extracted data will then be organized into collections of receipts defined by the user.

Using these extracted values, the system will compute summary statistics for each collection, including the average price per gallon, the total amount spent, the number of receipts in the collection, and the date range represented by the receipts.

This project is intended as a proof-of-concept system designed to demonstrate the feasibility of combining OCR technology with simple automated parsing techniques in a Python environment. The application is not intended to serve as a production-ready financial system, but rather as a prototype illustrating how automation tools can assist small single-user businesses with basic expense analysis.

## 1.2 Roles

The development team consists of the following members:

James Green

Responsible for system planning, requirements development, implementation of core application functionality, and documentation of system behavior.

Chris Duckers

Responsible for system design development, review of requirements and implementation artifacts, and collaboration on application development.

Numi

Responsible for creating and maintaining documentation, presentation, and users guide.

All team members will participate in system testing, review activities, and documentation refinement throughout the project lifecycle.

## 1.3 Deliverables

The project will produce the following deliverables:

### 1.3.1 Planning Document

Provides an overview of the project goals, objectives, and development strategy. This document outlines the development timeline, team responsibilities, tools and technologies used, and project assumptions. It serves as the foundation for all subsequent development activities and ensures that the team has a clear understanding of the project scope and direction

### 1.3.2 Software Requirements Specification (SRS)

Defines the functional and non-functional requirements of the Receipt Scanner application. This document describes the expected behavior of the system, the features it must provide, and the constraints under which it must operate. The SRS serves as a formal agreement between developers and reviewers regarding what the system must accomplish.

### 1.3.3 Software Design Description (SWDD)

Outlines the architectural and technical design of the Receipt Scanner system. This document explains how the system components interact, the structure of the software modules, and the technologies used to implement the application. The design document provides detailed technical information necessary for implementing and maintaining the system.

### 1.3.4 Application Implementation

Consists of the fully developed Receipt Scanner software written in Python. The application will include modules responsible for image upload, OCR processing, data extraction, receipt organization, and statistical analysis.

### 1.3.5 User Guide

Provides instructions for installing, running, and using the Receipt Scanner system. It explains how users can upload receipt images, create receipt collections, view extracted information, and analyze summary statistics.

# CHAPTER 2 DEVELOPMENT

## 2.1 Assumptions

The following assumptions apply to the development and operation of the Receipt Scanner system:

- Users will provide receipt images in a clear and readable format suitable for OCR processing.
- Receipts are assumed to be standard retail receipts containing recognizable textual patterns such as vendor names, dates, and totals.
- The system will be used by a single user operating locally on their machine.
- Browser storage may impose limitations on the number of receipts that can be loaded simultaneously.
- Uploaded files will be validated to confirm they are image files before processing.
- Security concerns related to storing uploaded images locally are considered out of scope for this proof-of-concept implementation.
- Receipt collections will typically contain fewer than 100 receipts.

## 2.2 Tools and Languages

1. The Receipt Scanner system will be implemented entirely in Python.
2. The following technologies and tools will be used during development:
3. Flask python web framework will be used to host a local web server and provide the browser-based interface.
4. **OCR Processing**

   An open-source OCR engine such as **Tesseract OCR** will be used to extract text from uploaded receipt images.

5. **Image Processing**

   Python libraries such as **Pillow** or **OpenCV** may be used for image preprocessing prior to OCR extraction.

6. **Frontend Interface**

   The application will provide a simple browser-based interface allowing users to upload receipts, manage receipt collections, and view extracted data summaries.

7. Receipt images will be stored on the local file system, while extracted receipt metadata will be stored within the browser session for display and analysis.
8. All tools used in this project will be open source.

## 2.1 Version Control

Source code and documentation for the Receipt Scanner system will be maintained using a version control system such as Git. Version control will be used to track changes to the codebase, manage revisions to project documentation, and support collaborative development between team members.

# SOFTWARE REQUIREMENTS

## RECEIPT SCANNER VOL 2

This Page Intentionally Left Blank

# CHAPTER 3
# INTRODUCTION

## 3.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Receipt Scanner application.

The purpose of this document is to define the behavior and capabilities of the system in a clear and structured manner so that developers and reviewers can understand the expected functionality of the application prior to implementation.

## 3.2 Scope

The Receipt Scanner system is a Python-based application designed to digitize receipt images and extract structured information from them using Optical Character Recognition (OCR).

The system allows a user to create named collections of receipts, upload receipt images, extract relevant transaction information, and compute summary statistics based on the extracted values.

The application will operate locally and will provide a browser-based interface for interacting with the system. The system is intended as a proof-of-concept prototype and is not designed for enterprise or multi-user deployment.

## 3.3 System Overview

The system provides a lightweight workflow for organizing and analyzing receipt data.

Users can create collections that represent groups of receipts. Receipts may then be uploaded individually or in batches. Each uploaded image will be processed using OCR to extract textual content. The system will then attempt to identify specific receipt fields such as vendor name, transaction date, price paid, and gallons purchased.

After extraction, the system will display the extracted information and allow the user to manually edit any fields if the automatic extraction is incorrect. Receipts flagged as problematic by the extraction process will be displayed separately and excluded from summary calculations.

The system will compute several summary statistics for each collection, including:

- Average price per gallon
- Total amount spent
- Date range of receipts
- Number of valid receipts in the collection

These summaries allow users such as small single-operator businesses to quickly analyze vehicle fuel expenses.

# CHAPTER 4
# FUNCTIONAL REQUIREMENTS

## 4.1 Upload Receipt Image

**Table 2-1 Receipt Upload Requirements**

| | |
|---|---|
| **FR1-1** | Receipt Scanner **shall** support the following image formats<br>➢ JPG<br>➢ PNG<br>➢ HEIC |
| **FR1-2** | Receipt Scanner **shall** process each receipt individual after upload. |

## 4.2 Store Uploaded File

**Table 2-2 Receipt Storage Requirements**

| | |
|---|---|
| **FR2-1** | Receipt Scanner **shall** store uploaded receipt images. |
| **FR2-2** | Receipt Scanner **shall** verify that uploaded files are valid image formats. |

## 4.3 Perform OCR

**Table 2-3 Text Recognition Requirements**

| | |
|---|---|
| **FR3-1** | Receipt Scanner **shall** perform Optical Character Recognition on each uploaded receipt image. |

## 4.4 Extract Receipt Fields

**Table 2-4 Image Processing Requirements**

| | |
|---|---|
| **FR4-1** | Receipt Scanner **shall** attempt to identify the fields from the OCR output:<br>➢ Vendor name<br>➢ Transaction date<br>➢ Total price paid |
| **FR4-2** | Receipt Scanner **shall** run automatically after an image is uploaded. |

## 4.5 Display Extracted Results

**Table 2-5 Display Requirements**

| | |
|---|---|
| **FR5-1** | Receipt Scanner **shall** display the extracted receipt information to the user. |
| **FR5-2** | Receipt Scanner **shall** display summary statistics for the receipt collection. |

## 4.6 Allow Manual Correction

**Table 2-6 Display Requirements**

| | |
|---|---|
| **FR6-1** | Receipt Scanner **shall** allow the user to manually edit extracted receipt fields. |

## 4.7 Save or Export Results

**Table 2-7 Save and Export Requirements**

| | |
|---|---|
| **FR7-1** | Receipt Scanner **shall** retain receipt data within the application interface for the current session. |

# CHAPTER 5
# INTERFACE REQUIREMENTS

## 5.1 User Interface

**Table 3-1 User Interface Requirements**

| | |
|---|---|
| **IR1-1** | Receipt Scanner **shall** provide a browser-based interface allowing users to:<br>➢ Upload receipt images<br>➢ View extracted receipt data<br>➢ Edit extracted fields<br>➢ View summary statistics |

## 5.2 Software Interfaces

**Table 3-2 Software Interface Requirements**

| | |
|---|---|
| **IR2-1** | Receipt Scanner **shall** interact with an OCR engine to convert receipt images into machine-readable text. |

# CHAPTER 6
# NON-FUNCTIONAL REQUIREMENTS

## 6.1 Usability

**Table 4-1 Usability Requirements**

| | |
|---|---|
| **NFR-1-1** | Receipt Scanner **shall** provide a simple and intuitive web interface suitable for users with minimal technical experience. |

## 6.2 Reliability

**Table 4-2 Reliability Requirements**

| | |
|---|---|
| **NFR-2-1** | Receipt Scanner **shall** identify receipts that fail field extraction and flag them as problematic rather than using them in summary calculations. |

## 6.3 Maintainability

**Table 4-3 Maintainability Requirements**

| | |
|---|---|
| **NFR-3-1** | Receipt Scanner **shall** be implemented in modular Python components to allow future improvements to OCR processing or field extraction logic. |

# CHAPTER 7
# ACADEMIC REQUIREMENTS

## 7.1 Assignment Requirements

**Table 5-1 Assignment Requirements**

| | |
|---|---|
| **AR-5-1** | Receipt Scanner **shall** utilize Natural Language Processing, AI, or Intelligent Image Processing. |
| **AR-5-2** | Each Group Member **shall** participate in the coding process. |

## 7.2 External Requirements

**Table 5-1 Assignment Requirements**

| | |
|---|---|
| **AER-5-1** | All external libraries utilized will be specifically called out and included on the presentation |
| **AER-5-2** | Documentation for the project **shall** include all aspects of planning, documentation, and usability. |

# SOFTWARE DESIGN

## RECEIPT SCANNER VOL 3

This Page Intentionally Left Blank

# CHAPTER 1
# INTRODUCTION

## 1.1 Purpose

This Software Design Description (SWDD) defines the internal architecture and design of the Receipt Scanner system. The purpose of this document is to describe how the system implements the functional and non-functional requirements defined in the Software Requirements Specification.

## 1.2 Scope

This document describes the design of the Receipt Scanner software system, which is implemented using Python. The design focuses primarily on the internal processing engine responsible for receipt analysis and data extraction.

The system supports the following primary capabilities:

- ingestion of receipt images
- OCR extraction of text from images
- parsing of receipt fields
- validation and flagging of problematic receipts
- organization of receipts into collections
- generation of summary statistics for collections

External interfaces such as browser-based frontends are treated as adapters that invoke the core Python services. The SWDD therefore focuses on the design of the internal processing components rather than the implementation of any particular user interface.

# CHAPTER 2
# SYSTEM ARCHITECTURE

The receipt Scanner system design uses a modular architecture that allows the separation of the receipt's core functionality from the user interface. This allows the system to solely focus on the core problem which is extracting structured data from the receipt images while also being flexible in how the results are used.

The system architecture consists of the following components who will perform a specific task in the receipt processing pipeline:

- Image input Module
- OCR Processing Module
- Text Processing and Classification Module
- Data Structuring Module
- Interface Layer

The modular design ensure that the main processing engine will function independently from the interface used to display the results. For example, the structured data receipt can be presented via web interface, exported to a file for further analyst or presented via command line interface.

# CHAPTER 3
# SYSTEM DESIGN OVERVIEW

## 3.1 Design Objectives

The design of the Receipt Scanner system is guided by the following objectives:

### 3.1.1 Modularity

The system is structured as a collection of small, focused components responsible for distinct processing tasks such as OCR extraction, receipt parsing, and summary generation.

### 3.1.2 Interface Independence

The core processing logic is independent of any particular external interface. This allows the system to be accessed through different interfaces such as a web frontend, command-line interface, or automated batch process.

### 3.1.3 Maintainability

The system uses idiomatic Python programming practices and clear component boundaries to allow future improvements or modifications without requiring significant redesign.

### 3.1.4 Extensibility

The design allows core components such as the OCR processor or receipt parser to be replaced with alternate implementations without affecting other parts of the system.

### 3.1.5 Simplicity

Because the system is intended as a proof-of-concept prototype, the architecture prioritizes simplicity and clarity over complex infrastructure or enterprise-scale design patterns.

## 3.2 Architectural Overview

The Receipt Scanner system follows a modular layered architecture composed of the following major layers:

### 3.2.1 Interface Layer

External systems such as a web browser interface interact with the application through defined service interfaces.

### 3.2.2 Application Service Layer

Coordinates receipt ingestion, OCR processing, parsing, validation, and collection summary generation.

### 3.2.3 Domain Processing Layer

Contains the core logic for receipt parsing, validation, and data aggregation.

### 3.2.4 Infrastructure Layer

Provides services for OCR processing, image loading, and storage of receipt data.

The web interface does not directly perform any processing logic. Instead, it communicates with the application service layer, which orchestrates the processing workflow.

# CHAPTER 4
# COMPONENT DESIGN

The core system is composed of several primary software components. Each component is responsible for a specific function within the receipt processing pipeline.

## 4.1 Receipt Ingestion Component

The Receipt Ingestion Component is responsible for receiving receipt images from external interfaces and preparing them for processing.

Responsibilities include:

- accepting uploaded image files
- validating file format
- storing receipt images in the local filesystem
- initiating the receipt processing workflow

Supported input formats include:

- JPG
- PNG
- HEIC

If a file does not match the supported image formats, the ingestion component rejects the file before processing.

# CHAPTER 5
# DATA DESIGN

# CHAPTER 6
# PROCESSING WORKFLOWS

# CHAPTER 7
# ERROR HANDLING AND CONSTRAINTS

## 7.1 Error Handling

## 7.2 Design Constraints

# USER GUIDE

## RECEIPT SCANNER VOL 5

This Page Intentionally Left Blank

# CHAPTER 1
# INTRODUCTION

## 1.1 .

.